

LLaMA.cpp HTTP Server

```
/*
notes: 杨小兵-2024-12-16

1、这个项目是llama.cpp项目的子项目HTTP Server
*/
```

Fast, lightweight, pure C/C++ HTTP server based on [httplib](#), [nlohmann::json](#) and **llama.cpp**.

```
/*
notes: 杨小兵-2024-12-16

1、快速的、轻量级的、纯C/C++ HTTP server
2、这个HTTP server依赖httplib、json、llama.cpp这三个库
3、上述提到的httplib是我没有接触过的
*/
```

Set of LLM REST APIs and a simple web front end to interact with llama.cpp.

```
/*
notes: 杨小兵-2024-12-16

1、一组 LLM REST API 和一个用于与 llama.cpp 交互的简单 Web 前端。
2、这里有一组LLM REST API、web前端
*/
```

Features:

- LLM inference of F16 and quantized models on GPU and CPU
- [OpenAI API](#) compatible chat completions and embeddings routes
- Reranking endpoint (WIP: <https://github.com/ggerganov/llama.cpp/pull/9510>)
- Parallel decoding with multi-user support
- Continuous batching
- Multimodal (wip)
- Monitoring endpoints
- Schema-constrained JSON response format

```
/*
notes: 杨小兵-2024-12-16

1、将float-pointing 16和量化的LLM在GPU和CPU上进行inference
2、[OpenAI API](https://github.com/openai/openai-openapi) 兼容聊天完成和嵌入路线
```

```
3、Reranking endpoint ( 这个目前不了解 )
4、支持多用户并行解码
5、连续的batching
6、多模态
7、对endpoints进行监督
8、可以使用schema来限制JSON的回复格式
*/
```

The project is under active development, and we are [looking for feedback and contributors](#).

```
/*
notes:杨小兵-2024-12-16

1、这个项目还处于积极开发状态，想要获取一些反馈和贡献者
*/
```

1 Usage

```
/*
notes:杨小兵-2024-12-16

1、这部分注释是为了提示贡献者，下列的列表通过llama-gen-docs生成得到的
*/
```

Common params

Argument	Explanation
-h, --help, --usage	print usage and exit
--version	show version and build info
--verbose-prompt	print a verbose prompt before generation (default: false)
-t, --threads N	number of threads to use during generation (default: -1) (env: LLAMA_ARG_THREADS)
-tb, --threads-batch N	number of threads to use during batch and prompt processing (default: same as --threads)
-C, --cpu-mask M	CPU affinity mask: arbitrarily long hex. Complements cpu-range (default: "")
-Cr, --cpu-range lo-hi	range of CPUs for affinity. Complements --cpu-mask
--cpu-strict <0\ 1>	use strict CPU placement (default: 0)
--prio N	set process/thread priority : 0-normal, 1-medium, 2-high, 3-realtime (default: 0)

Argument	Explanation
<code>--poll <0...100></code>	use polling level to wait for work (0 - no polling, default: 50)
<code>-Cb, --cpu-mask-batch M</code>	CPU affinity mask: arbitrarily long hex. Complements cpu-range-batch (default: same as --cpu-mask)
<code>-Crb, --cpu-range-batch lo-hi</code>	ranges of CPUs for affinity. Complements --cpu-mask-batch
<code>--cpu-strict-batch <0\ 1></code>	use strict CPU placement (default: same as --cpu-strict)
<code>--prio-batch N</code>	set process/thread priority : 0-normal, 1-medium, 2-high, 3-realtime (default: 0)
<code>--poll-batch <0\ 1></code>	use polling to wait for work (default: same as --poll)
<code>-c, --ctx-size N</code>	size of the prompt context (default: 4096, 0 = loaded from model) (env: LLAMA_ARG_CTX_SIZE)
<code>-n, --predict, --n-predict N</code>	number of tokens to predict (default: -1, -1 = infinity, -2 = until context filled) (env: LLAMA_ARG_N_PREDICT)
<code>-b, --batch-size N</code>	logical maximum batch size (default: 2048) (env: LLAMA_ARG_BATCH)
<code>-ub, --ubatch-size N</code>	physical maximum batch size (default: 512) (env: LLAMA_ARG_UBATCH)
<code>--keep N</code>	number of tokens to keep from the initial prompt (default: 0, -1 = all)
<code>-fa, --flash-attn</code>	enable Flash Attention (default: disabled) (env: LLAMA_ARG_FLASH_ATTN)
<code>-p, --prompt PROMPT</code>	prompt to start generation with
<code>--no-perf</code>	disable internal libllama performance timings (default: false) (env: LLAMA_ARG_NO_PERF)
<code>-f, --file FNAME</code>	a file containing the prompt (default: none)
<code>-bf, --binary-file FNAME</code>	binary file containing the prompt (default: none)
<code>-e, --escape</code>	process escapes sequences (\n, \r, \t, ', ", \) (default: true)
<code>--no-escape</code>	do not process escape sequences
<code>--rope-scaling {none,linear,yarn}</code>	RoPE frequency scaling method, defaults to linear unless specified by the model (env: LLAMA_ARG_ROPE_SCALING_TYPE)
<code>--rope-scale N</code>	RoPE context scaling factor, expands context by a factor of N (env: LLAMA_ARG_ROPE_SCALE)

Argument	Explanation
<code>--rope-freq-base N</code>	RoPE base frequency, used by NTK-aware scaling (default: loaded from model) (env: LLAMA_ARG_ROPE_FREQ_BASE)
<code>--rope-freq-scale N</code>	RoPE frequency scaling factor, expands context by a factor of 1/N (env: LLAMA_ARG_ROPE_FREQ_SCALE)
<code>--yarn-orig-ctx N</code>	YaRN: original context size of model (default: 0 = model training context size) (env: LLAMA_ARG_YARN_ORIG_CTX)
<code>--yarn-ext-factor N</code>	YaRN: extrapolation mix factor (default: -1.0, 0.0 = full interpolation) (env: LLAMA_ARG_YARN_EXT_FACTOR)
<code>--yarn-attn-factor N</code>	YaRN: scale sqrt(t) or attention magnitude (default: 1.0) (env: LLAMA_ARG_YARN_ATTN_FACTOR)
<code>--yarn-beta-slow N</code>	YaRN: high correction dim or alpha (default: 1.0) (env: LLAMA_ARG_YARN_BETA_SLOW)
<code>--yarn-beta-fast N</code>	YaRN: low correction dim or beta (default: 32.0) (env: LLAMA_ARG_YARN_BETA_FAST)
<code>-dkvc, --dump-kv-cache</code>	verbose print of the KV cache
<code>-nkvo, --no-kv-offload</code>	disable KV offload (env: LLAMA_ARG_NO_KV_OFFLOAD)
<code>-ctk, --cache-type-k TYPE</code>	KV cache data type for K allowed values: f32, f16, bf16, q8_0, q4_0, q4_1, iq4_nl, q5_0, q5_1 (default: f16) (env: LLAMA_ARG_CACHE_TYPE_K)
<code>-ctv, --cache-type-v TYPE</code>	KV cache data type for V allowed values: f32, f16, bf16, q8_0, q4_0, q4_1, iq4_nl, q5_0, q5_1 (default: f16) (env: LLAMA_ARG_CACHE_TYPE_V)
<code>-dt, --defrag-thold N</code>	KV cache defragmentation threshold (default: 0.1, < 0 - disabled) (env: LLAMA_ARG_DEFRAG_THOLD)
<code>-np, --parallel N</code>	number of parallel sequences to decode (default: 1) (env: LLAMA_ARG_N_PARALLEL)
<code>--mlock</code>	force system to keep model in RAM rather than swapping or compressing (env: LLAMA_ARG_MLOCK)
<code>--no-mmap</code>	do not memory-map model (slower load but may reduce pageouts if not using mlock) (env: LLAMA_ARG_NO_MMAP)

Argument	Explanation
<code>--numa TYPE</code>	attempt optimizations that help on some NUMA systems - distribute: spread execution evenly over all nodes - isolate: only spawn threads on CPUs on the node that execution started on - numactl: use the CPU map provided by numactl if run without this previously, it is recommended to drop the system page cache before using this see https://github.com/ggerganov/llama.cpp/issues/1437 (env: LLAMA_ARG_NUMA)
<code>-dev, --device <dev1,dev2,...></code>	comma-separated list of devices to use for offloading (none = don't offload) use <code>--list-devices</code> to see a list of available devices (env: LLAMA_ARG_DEVICE)
<code>--list-devices</code>	print list of available devices and exit
<code>-ngl, --gpu-layers, --n-gpu-layers N</code>	number of layers to store in VRAM (env: LLAMA_ARG_N_GPU_LAYERS)
<code>-sm, --split-mode {none,layer,row}</code>	how to split the model across multiple GPUs, one of: - none: use one GPU only - layer (default): split layers and KV across GPUs - row: split rows across GPUs (env: LLAMA_ARG_SPLIT_MODE)
<code>-ts, --tensor-split N0,N1,N2,...</code>	fraction of the model to offload to each GPU, comma-separated list of proportions, e.g. 3,1 (env: LLAMA_ARG_TENSOR_SPLIT)
<code>-mg, --main-gpu INDEX</code>	the GPU to use for the model (with split-mode = none), or for intermediate results and KV (with split-mode = row) (default: 0) (env: LLAMA_ARG_MAIN_GPU)
<code>--check-tensors</code>	check model tensor data for invalid values (default: false)
<code>--override-kv KEY=TYPE:VALUE</code>	advanced option to override model metadata by key. may be specified multiple times. types: int, float, bool, str. example: <code>--override-kv tokenizer.ggml.add_bos_token=bool:false</code>
<code>--lora FNAME</code>	path to LoRA adapter (can be repeated to use multiple adapters)
<code>--lora-scaled FNAME SCALE</code>	path to LoRA adapter with user defined scaling (can be repeated to use multiple adapters)
<code>--control-vector FNAME</code>	add a control vector note: this argument can be repeated to add multiple control vectors
<code>--control-vector-scaled FNAME SCALE</code>	add a control vector with user defined scaling SCALE note: this argument can be repeated to add multiple scaled control vectors

Argument	Explanation
<code>--control-vector-layer-range START END</code>	layer range to apply the control vector(s) to, start and end inclusive
<code>-m, --model FNAME</code>	model path (default: <code>models/\$filename</code> with filename from <code>--hf-file</code> or <code>--model-url</code> if set, otherwise <code>models/7B/ggml-model-f16.gguf</code>) (env: LLAMA_ARG_MODEL)
<code>-mu, --model-url MODEL_URL</code>	model download url (default: unused) (env: LLAMA_ARG_MODEL_URL)
<code>-hfr, --hf-repo REPO</code>	Hugging Face model repository (default: unused) (env: LLAMA_ARG_HF_REPO)
<code>-hff, --hf-file FILE</code>	Hugging Face model file (default: unused) (env: LLAMA_ARG_HF_FILE)
<code>-hft, --hf-token TOKEN</code>	Hugging Face access token (default: value from HF_TOKEN environment variable) (env: HF_TOKEN)
<code>--log-disable</code>	Log disable
<code>--log-file FNAME</code>	Log to file
<code>--log-colors</code>	Enable colored logging (env: LLAMA_LOG_COLORS)
<code>-v, --verbose, --log-verbose</code>	Set verbosity level to infinity (i.e. log all messages, useful for debugging)
<code>-lv, --verbosity, --log-verbosity N</code>	Set the verbosity threshold. Messages with a higher verbosity will be ignored. (env: LLAMA_LOG_VERBOSITY)
<code>--log-prefix</code>	Enable prefix in log messages (env: LLAMA_LOG_PREFIX)
<code>--log-timestamps</code>	Enable timestamps in log messages (env: LLAMA_LOG_TIMESTAMPS)

Sampling params

Argument	Explanation
<code>--samplers SAMPLERS</code>	samplers that will be used for generation in the order, separated by ';' (default: dry;top_k;typ_p;top_p;min_p;xtc;temperature)
<code>-s, --seed SEED</code>	RNG seed (default: -1, use random seed for -1)
<code>--sampling-seq SEQUENCE</code>	simplified sequence for samplers that will be used (default: dkypmxt)

Argument	Explanation
<code>--ignore-eos</code>	ignore end of stream token and continue generating (implies <code>--logit-bias EOS-inf</code>)
<code>--penalize-nl</code>	penalize newline tokens (default: false)
<code>--temp N</code>	temperature (default: 0.8)
<code>--top-k N</code>	top-k sampling (default: 40, 0 = disabled)
<code>--top-p N</code>	top-p sampling (default: 0.9, 1.0 = disabled)
<code>--min-p N</code>	min-p sampling (default: 0.1, 0.0 = disabled)
<code>--xtc-probability N</code>	xtc probability (default: 0.0, 0.0 = disabled)
<code>--xtc-threshold N</code>	xtc threshold (default: 0.1, 1.0 = disabled)
<code>--typical N</code>	locally typical sampling, parameter p (default: 1.0, 1.0 = disabled)
<code>--repeat-last-n N</code>	last n tokens to consider for penalize (default: 64, 0 = disabled, -1 = ctx_size)
<code>--repeat-penalty N</code>	penalize repeat sequence of tokens (default: 1.0, 1.0 = disabled)
<code>--presence-penalty N</code>	repeat alpha presence penalty (default: 0.0, 0.0 = disabled)
<code>--frequency-penalty N</code>	repeat alpha frequency penalty (default: 0.0, 0.0 = disabled)
<code>--dry-multiplier N</code>	set DRY sampling multiplier (default: 0.0, 0.0 = disabled)
<code>--dry-base N</code>	set DRY sampling base value (default: 1.75)
<code>--dry-allowed-length N</code>	set allowed length for DRY sampling (default: 2)
<code>--dry-penalty-last-n N</code>	set DRY penalty for the last n tokens (default: -1, 0 = disable, -1 = context size)
<code>--dry-sequence-breaker STRING</code>	add sequence breaker for DRY sampling, clearing out default breakers (<code>"\n", ':", "'", '*'</code>) in the process; use "none" to not use any sequence breakers
<code>--dynatemp-range N</code>	dynamic temperature range (default: 0.0, 0.0 = disabled)
<code>--dynatemp-exp N</code>	dynamic temperature exponent (default: 1.0)
<code>--mirostat N</code>	use Mirostat sampling. Top K, Nucleus and Locally Typical samplers are ignored if used. (default: 0, 0 = disabled, 1 = Mirostat, 2 = Mirostat 2.0)
<code>--mirostat-lr N</code>	Mirostat learning rate, parameter eta (default: 0.1)
<code>--mirostat-ent N</code>	Mirostat target entropy, parameter tau (default: 5.0)

Argument	Explanation
<code>-l, --logit-bias TOKEN_ID(+/-)BIAS</code>	modifies the likelihood of token appearing in the completion, i.e. <code>--logit-bias 15043+1</code> to increase likelihood of token ' Hello', or <code>--logit-bias 15043-1</code> to decrease likelihood of token ' Hello'
<code>--grammar GRAMMAR</code>	BNF-like grammar to constrain generations (see samples in grammars/ dir) (default: "")
<code>--grammar-file FNAME</code>	file to read grammar from
<code>-j, --json-schema SCHEMA</code>	JSON schema to constrain generations (https://json-schema.org/), e.g. <code>{}</code> for any JSON object For schemas w/ external \$refs, use <code>--grammar + example/json_schema_to_grammar.py</code> instead

Example-specific params

Argument	Explanation
<code>--no-context-shift</code>	disables context shift on infinite text generation (default: disabled) (env: LLAMA_ARG_NO_CONTEXT_SHIFT)
<code>-sp, --special</code>	special tokens output enabled (default: false)
<code>--no-warmup</code>	skip warming up the model with an empty run
<code>--spm-infill</code>	use Suffix/Prefix/Middle pattern for infill (instead of Prefix/Suffix/Middle) as some models prefer this. (default: disabled)
<code>--pooling {none,mean,cls,last,rank}</code>	pooling type for embeddings, use model default if unspecified (env: LLAMA_ARG_POOLING)
<code>-cb, --cont-batching</code>	enable continuous batching (a.k.a dynamic batching) (default: enabled) (env: LLAMA_ARG_CONT_BATCHING)
<code>-nocb, --no-cont-batching</code>	disable continuous batching (env: LLAMA_ARG_NO_CONT_BATCHING)
<code>-a, --alias STRING</code>	set alias for model name (to be used by REST API) (env: LLAMA_ARG_ALIAS)
<code>--host HOST</code>	ip address to listen (default: 127.0.0.1) (env: LLAMA_ARG_HOST)
<code>--port PORT</code>	port to listen (default: 8080) (env: LLAMA_ARG_PORT)
<code>--path PATH</code>	path to serve static files from (default:) (env: LLAMA_ARG_STATIC_PATH)
<code>--no-webui</code>	Disable the Web UI (default: enabled) (env: LLAMA_ARG_NO_WEBUI)

Argument	Explanation
<code>--embedding, --embeddings</code>	restrict to only support embedding use case; use only with dedicated embedding models (default: disabled) (env: LLAMA_ARG_EMBEDDINGS)
<code>--reranking, --rerank</code>	enable reranking endpoint on server (default: disabled) (env: LLAMA_ARG_RERANKING)
<code>--api-key KEY</code>	API key to use for authentication (default: none) (env: LLAMA_API_KEY)
<code>--api-key-file FNAME</code>	path to file containing API keys (default: none)
<code>--ssl-key-file FNAME</code>	path to file a PEM-encoded SSL private key (env: LLAMA_ARG_SSL_KEY_FILE)
<code>--ssl-cert-file FNAME</code>	path to file a PEM-encoded SSL certificate (env: LLAMA_ARG_SSL_CERT_FILE)
<code>-to, --timeout N</code>	server read/write timeout in seconds (default: 600) (env: LLAMA_ARG_TIMEOUT)
<code>--threads-http N</code>	number of threads used to process HTTP requests (default: -1) (env: LLAMA_ARG_THREADS_HTTP)
<code>--cache-reuse N</code>	min chunk size to attempt reusing from the cache via KV shifting (default: 0) (env: LLAMA_ARG_CACHE_REUSE)
<code>--metrics</code>	enable prometheus compatible metrics endpoint (default: disabled) (env: LLAMA_ARG_ENDPOINT_METRICS)
<code>--slots</code>	enable slots monitoring endpoint (default: disabled) (env: LLAMA_ARG_ENDPOINT_SLOTS)
<code>--props</code>	enable changing global properties via POST /props (default: disabled) (env: LLAMA_ARG_ENDPOINT_PROPS)
<code>--no-slots</code>	disables slots monitoring endpoint (env: LLAMA_ARG_NO_ENDPOINT_SLOTS)
<code>--slot-save-path PATH</code>	path to save slot kv cache (default: disabled)

Argument	Explanation
<code>--chat-template</code> <code>JINJA_TEMPLATE</code>	set custom jinja chat template (default: template taken from model's metadata) if suffix/prefix are specified, template will be disabled list of built-in templates: chatglm3, chatglm4, chatml, command-r, deepseek, deepseek2, exaone3, gemma, granite, llama2, llama2-sys, llama2-sys-bos, llama2-sys-strip, llama3, minicpm, mistral-v1, mistral-v3, mistral-v3-tekken, mistral-v7, monarch, openchat, orion, phi3, rwkv-world, vicuna, vicuna-orca, zephyr (env: LLAMA_ARG_CHAT_TEMPLATE)
<code>-sps, --slot-prompt-similarity SIMILARITY</code>	how much the prompt of a request must match the prompt of a slot in order to use that slot (default: 0.50, 0.0 = disabled)
<code>--lora-init-without-apply</code>	load LoRA adapters without applying them (apply later via POST /lora-adapters) (default: disabled)
<code>--draft-max, --draft, --draft-n N</code>	number of tokens to draft for speculative decoding (default: 16) (env: LLAMA_ARG_DRAFT_MAX)
<code>--draft-min, --draft-n-min N</code>	minimum number of draft tokens to use for speculative decoding (default: 5) (env: LLAMA_ARG_DRAFT_MIN)
<code>--draft-p-min P</code>	minimum speculative decoding probability (greedy) (default: 0.9) (env: LLAMA_ARG_DRAFT_P_MIN)
<code>-cd, --ctx-size-draft N</code>	size of the prompt context for the draft model (default: 0, 0 = loaded from model) (env: LLAMA_ARG_CTX_SIZE_DRAFT)
<code>-devd, --device-draft <dev1,dev2,...></code>	comma-separated list of devices to use for offloading the draft model (none = don't offload) use --list-devices to see a list of available devices
<code>-ngld, --gpu-layers-draft, --n-gpu-layers-draft N</code>	number of layers to store in VRAM for the draft model (env: LLAMA_ARG_N_GPU_LAYERS_DRAFT)
<code>-md, --model-draft FNAME</code>	draft model for speculative decoding (default: unused) (env: LLAMA_ARG_MODEL_DRAFT)

Note: If both command line argument and environment variable are both set for the same param, the argument will take precedence over env var.

```
/*
notes:杨小兵-2024-12-16
```

```
1、注意：如果命令行参数和环境变量都为同一个参数设置，则参数将优先于环境变量。  
*/
```

Example usage of docker compose with environment variables:

```
/*  
notes:杨小兵-2024-12-16  
  
1、使用环境变量的docker compose的示例  
*/
```

```
services:  
  llamacpp-server:  
    image: ghcr.io/ggerganov/llama.cpp:server  
    ports:  
      - 8080:8080  
    volumes:  
      - ./models:/models  
    environment:  
      # alternatively, you can use "LLAMA_ARG_MODEL_URL" to download the model  
      LLAMA_ARG_MODEL: /models/my_model.gguf  
      LLAMA_ARG_CTX_SIZE: 4096  
      LLAMA_ARG_N_PARALLEL: 2  
      LLAMA_ARG_ENDPOINT_METRICS: 1  
      LLAMA_ARG_PORT: 8080
```

2 Build

llama-server is built alongside everything else from the root of the project

```
/*  
notes:杨小兵-2024-12-16  
  
1、这个部分为了解释如何构建llama.cpp项目中的llama-server子项目  
2、“llama-server” 与项目根目录中的其他所有内容一起构建  
*/
```

- Using **CMake**:

```
cmake -B build  
cmake --build build --config Release -t llama-server
```

Binary is at **./build/bin/llama-server**

3 Build with SSL

`llama-server` can also be built with SSL support using OpenSSL 3

```
/*
notes: 杨小兵-2024-12-16

1、`llama-server` 也可以使用 OpenSSL 3 构建 SSL 支持
2、在构建llama-server的时候通过构建SSL的支持
3、下列在CMAKE中使用宏定义来控制编译
*/
```

- Using `CMake`:

```
cmake -B build -DLLAMA_SERVER_SSL=ON
cmake --build build --config Release -t llama-server
```

4 Web UI

The project includes a web-based user interface that enables interaction with the model through the `/chat/completions` endpoint.

```
/*
notes: 杨小兵-2024-12-16

1、该项目包括一个基于 Web 的用户界面，可通过“/chat/completions”端点与模型进行交互
2、这个项目包含：用户界面（这个用户界面可以通过上述提到的endpoint来同模型进行交互）
*/
```

The web UI is developed using:

- `vue` framework for frontend development
- `tailwindcss` and `daisyui` for styling
- `vite` for build tooling

```
/*
notes: 杨小兵-2024-12-17

1、这个web ui的开发使用到了三个工具：vue、tailwindcss、daisyui、vite
2、前端开发用到了vue框架
3、风格化的东西使用到了`tailwindcss` and `daisyui`
4、构建tooling使用到了vite
*/
```

A pre-built version is available as a single HTML file under `/public` directory.

```
/*
notes:杨小兵-2024-12-17

1、在/public目录下有一个单HTML文件是可以使用的，这个文件是预构建好的
*/
```

To build or to run the dev server (with hot reload):

```
# make sure you have nodejs installed
cd examples/server/webui
npm i

# to run the dev server
npm run dev

# to build the public/index.html
npm run build
```

NOTE: if you are using the vite dev server, you can change the API base URL to llama.cpp. To do that, run this code snippet in browser's console:

```
localStorage.setItem('base', 'http://localhost:8080')
```

```
/*
notes:杨小兵-2024-12-17

1、注意：如果您使用的是 vite dev 服务器，则可以将 API 基本 URL 更改为 llama.cpp。为此，
请在浏览器的控制台中运行此代码片段：
*/
```

5 Quick Start

To get started right away, run the following command, making sure to use the correct path for the model you have:

```
/*
notes:杨小兵-2024-12-17

1、确保模型的路径是正确的，这样可以使用llama-server
*/
```

5.1 Unix-based systems (Linux, macOS, etc.)

```
./llama-server -m models/7B/ggml-model.gguf -c 2048
```

5.2 Windows

```
llama-server.exe -m models\7B\ggml-model.gguf -c 2048
```

The above command will start a server that by default listens on `127.0.0.1:8080`. You can consume the endpoints with Postman or NodeJS with axios library. You can visit the web front end at the same url.

```
/*
notes:杨小兵-2024-12-17

1、这部分内容需要额外了解然后进行修改
*/
```

5.3 Docker

```
docker run -p 8080:8080 -v /path/to/models:/models
ghcr.io/ggerganov/llama.cpp:server -m models/7B/ggml-model.gguf -c 512 --host
0.0.0.0 --port 8080

# or, with CUDA:
docker run -p 8080:8080 -v /path/to/models:/models --gpus all
ghcr.io/ggerganov/llama.cpp:server-cuda -m models/7B/ggml-model.gguf -c 512 --host
0.0.0.0 --port 8080 --n-gpu-layers 99
```

6 Testing with CURL

Using [curl](#). On Windows, [curl.exe](#) should be available in the base OS.

```
curl --request POST \
  --url http://localhost:8080/completion \
  --header "Content-Type: application/json" \
  --data '{"prompt": "Building a website can be done in 10 simple
steps:", "n_predict": 128}'
```

```
/*
notes:杨小兵-2024-12-17
```

```
1、使用CURL来对llama-server进行测试，模仿其他用户使用llama-server的场景
*/
```

7 Advanced testing

We implemented a [server test framework](#) using human-readable scenario.

Before submitting an issue, please try to reproduce it with this format.

```
/*
notes:杨小兵-2024-12-17

1、使用人类可读的场景实现了一个服务测试框架
2、在提交一个issue的时候请尝试使用这种格式进行复现
*/
```

8 Node JS Test

You need to have [Node.js](#) installed.

```
mkdir llama-client
cd llama-client
```

Create an index.js file and put this inside:

```
const prompt = "Building a website can be done in 10 simple steps:"

async function test() {
  let response = await fetch("http://127.0.0.1:8080/completion", {
    method: "POST",
    body: JSON.stringify({
      prompt,
      n_predict: 64,
    })
  })
  console.log((await response.json()).content)
}

test()
```

And run it:

```
node index.js
```

```
/*
notes:杨小兵-2024-12-17

1、整体上为了测试Node JS
*/
```

9 API Endpoints

9.1 GET `/health`: Returns health check result

Response format

- HTTP status code 503
 - Body: `{"error": {"code": 503, "message": "Loading model", "type": "unavailable_error"}}`
 - Explanation: the model is still being loaded.
- HTTP status code 200
 - Body: `{"status": "ok" }`
 - Explanation: the model is successfully loaded and the server is ready.

```
/*
notes:杨小兵-2024-12-17

1、这部分内容不理解，我应该从哪里获取得到？
*/
```

9.2 POST `/completion`: Given a `prompt`, it returns the predicted completion.

```
/*
notes:杨小兵-2024-12-17

1、这部分内容介绍的是对模型的设置以及如何获取得到模型的输出
2、下列涉及到的一些参数我都是不理解的
*/
```

Options:

prompt: Provide the prompt for this completion as a string or as an array of strings or numbers representing tokens. Internally, if `cache_prompt` is `true`, the prompt is compared to the previous completion and only the "unseen" suffix is evaluated. A `BOS` token is inserted at the start, if all of the following conditions are true:

- The prompt is a string or an array with the first element given as a string
- The model's `tokenizer.ggml.add_bos_token` metadata is `true`

These input shapes and data type are allowed for **prompt**:

- Single string: `"string"`
- Single sequence of tokens: `[12, 34, 56]`
- Mixed tokens and strings: `[12, 34, "string", 56, 78]`

Multiple prompts are also supported. In this case, the completion result will be an array.

- Only strings: `["string1", "string2"]`
- Strings and sequences of tokens: `["string1", [12, 34, 56]]`
- Mixed types: `[[12, 34, "string", 56, 78], [12, 34, 56], "string"]`

temperature: Adjust the randomness of the generated text. Default: `0.8`

dynatemp_range: Dynamic temperature range. The final temperature will be in the range of `[temperature - dynatemp_range; temperature + dynatemp_range]` Default: `0.0`, which is disabled.

dynatemp_exponent: Dynamic temperature exponent. Default: `1.0`

top_k: Limit the next token selection to the K most probable tokens. Default: `40`

top_p: Limit the next token selection to a subset of tokens with a cumulative probability above a threshold P. Default: `0.95`

min_p: The minimum probability for a token to be considered, relative to the probability of the most likely token. Default: `0.05`

n_predict: Set the maximum number of tokens to predict when generating text. **Note:** May exceed the set limit slightly if the last token is a partial multibyte character. When 0, no tokens will be generated but the prompt is evaluated into the cache. Default: `-1`, where `-1` is infinity.

n_indent: Specify the minimum line indentation for the generated text in number of whitespace characters. Useful for code completion tasks. Default: `0`

n_keep: Specify the number of tokens from the prompt to retain when the context size is exceeded and tokens need to be discarded. The number excludes the BOS token. By default, this value is set to `0`, meaning no tokens are kept. Use `-1` to retain all tokens from the prompt.

stream: Allows receiving each predicted token in real-time instead of waiting for the completion to finish (uses a different response format). To enable this, set to `true`.

stop: Specify a JSON array of stopping strings. These words will not be included in the completion, so make sure to add them to the prompt for the next iteration. Default: `[]`

typical_p: Enable locally typical sampling with parameter p. Default: `1.0`, which is disabled.

repeat_penalty: Control the repetition of token sequences in the generated text. Default: `1.1`

repeat_last_n: Last n tokens to consider for penalizing repetition. Default: `64`, where `0` is disabled and `-1` is ctx-size.

penalize_nl: Penalize newline tokens when applying the repeat penalty. Default: `true`

presence_penalty: Repeat alpha presence penalty. Default: `0.0`, which is disabled.

frequency_penalty: Repeat alpha frequency penalty. Default: **0.0**, which is disabled.

dry_multiplier: Set the DRY (Don't Repeat Yourself) repetition penalty multiplier. Default: **0.0**, which is disabled.

dry_base: Set the DRY repetition penalty base value. Default: **1.75**

dry_allowed_length: Tokens that extend repetition beyond this receive exponentially increasing penalty: multiplier * base ^ (length of repeating sequence before token - allowed length). Default: **2**

dry_penalty_last_n: How many tokens to scan for repetitions. Default: **-1**, where **0** is disabled and **-1** is context size.

dry_sequence_breakers: Specify an array of sequence breakers for DRY sampling. Only a JSON array of strings is accepted. Default: `['\n', ':', "'", '*']`

xtc_probability: Set the chance for token removal via XTC sampler. Default: **0.0**, which is disabled.

xtc_threshold: Set a minimum probability threshold for tokens to be removed via XTC sampler. Default: **0.1** (> **0.5** disables XTC)

mirostat: Enable Mirostat sampling, controlling perplexity during text generation. Default: **0**, where **0** is disabled, **1** is Mirostat, and **2** is Mirostat 2.0.

mirostat_tau: Set the Mirostat target entropy, parameter tau. Default: **5.0**

mirostat_eta: Set the Mirostat learning rate, parameter eta. Default: **0.1**

grammar: Set grammar for grammar-based sampling. Default: no grammar

json_schema: Set a JSON schema for grammar-based sampling (e.g. `{"items": {"type": "string"}, "minItems": 10, "maxItems": 100}` of a list of strings, or `{}` for any JSON). See [tests](#) for supported features. Default: no JSON schema.

seed: Set the random number generator (RNG) seed. Default: **-1**, which is a random seed.

ignore_eos: Ignore end of stream token and continue generating. Default: **false**

logit_bias: Modify the likelihood of a token appearing in the generated text completion. For example, use `"logit_bias": [[15043, 1.0]]` to increase the likelihood of the token 'Hello', or `"logit_bias": [[15043, -1.0]]` to decrease its likelihood. Setting the value to false, `"logit_bias": [[15043, false]]` ensures that the token **Hello** is never produced. The tokens can also be represented as strings, e.g. `["Hello, World!", -0.5]` will reduce the likelihood of all the individual tokens that represent the string **Hello, World!**, just like the **presence_penalty** does. Default: `[]`

n_probs: If greater than 0, the response also contains the probabilities of top N tokens for each generated token given the sampling settings. Note that for temperature < 0 the tokens are sampled greedily but token probabilities are still being calculated via a simple softmax of the logits without considering any other sampler settings. Default: **0**

min_keep: If greater than 0, force samplers to return N possible tokens at minimum. Default: **0**

t_max_predict_ms: Set a time limit in milliseconds for the prediction (a.k.a. text-generation) phase. The timeout will trigger if the generation takes more than the specified time (measured since the first token was generated) and if a new-line character has already been generated. Useful for FIM applications. Default: `0`, which is disabled.

image_data: An array of objects to hold base64-encoded image **data** and its **ids** to be reference in **prompt**. You can determine the place of the image in the prompt as in the following: `USER:[img-12]Describe the image in detail.\nASSISTANT:.` In this case, `[img-12]` will be replaced by the embeddings of the image with id `12` in the following **image_data** array: `{..., "image_data": [{"data": "<BASE64_STRING>", "id": 12}]}`. Use **image_data** only with multimodal models, e.g., LLaVA.

id_slot: Assign the completion task to an specific slot. If is `-1` the task will be assigned to a Idle slot. Default: `-1`

cache_prompt: Re-use KV cache from a previous request if possible. This way the common prefix does not have to be re-processed, only the suffix that differs between the requests. Because (depending on the backend) the logits are **not** guaranteed to be bit-for-bit identical for different batch sizes (prompt processing vs. token generation) enabling this option can cause nondeterministic results. Default: `true`

samplers: The order the samplers should be applied in. An array of strings representing sampler type names. If a sampler is not set, it will not be used. If a sampler is specified more than once, it will be applied multiple times. Default: `["dry", "top_k", "typ_p", "top_p", "min_p", "xtc", "temperature"]` - these are all the available values.

timings_per_token: Include prompt processing and text generation speed information in each response. Default: `false`

Response format

- Note: In streaming mode (**stream**), only **content** and **stop** will be returned until end of completion. Responses are sent using the **Server-sent events** standard. Note: the browser's **EventSource** interface cannot be used due to its lack of **POST** request support.
- completion_probabilities**: An array of token probabilities for each completion. The array's length is **n_predict**. Each item in the array has the following structure:

```
{
  "content": "<the token selected by the model>",
  "probs": [
    {
      "prob": float,
      "tok_str": "<most likely token>"
    },
    {
      "prob": float,
      "tok_str": "<second most likely token>"
    },
    ...
  ]
},
```

Notice that each `probs` is an array of length `n_probs`.

- `content`: Completion result as a string (excluding `stopping_word` if any). In case of streaming mode, will contain the next token as a string.
- `stop`: Boolean for use with `stream` to check whether the generation has stopped (Note: This is not related to stopping words array `stop` from input options)
- `generation_settings`: The provided options above excluding `prompt` but including `n_ctx`, `model`. These options may differ from the original ones in some way (e.g. bad values filtered out, strings converted to tokens, etc.).
- `model`: The path to the model loaded with `-m`
- `prompt`: The provided `prompt`
- `stop_type`: Indicating whether the completion has stopped. Possible values are:
 - `none`: Generating (not stopped)
 - `eos`: Stopped because it encountered the EOS token
 - `limit`: Stopped because `n_predict` tokens were generated before stop words or EOS was encountered
 - `word`: Stopped due to encountering a stopping word from `stop` JSON array provided
- `stopping_word`: The stopping word encountered which stopped the generation (or "" if not stopped due to a stopping word)
- `timings`: Hash of timing information about the completion such as the number of tokens `predicted_per_second`
- `tokens_cached`: Number of tokens from the prompt which could be re-used from previous completion (`n_past`)
- `tokens_evaluated`: Number of tokens evaluated in total from the prompt
- `truncated`: Boolean indicating if the context size was exceeded during generation, i.e. the number of tokens provided in the prompt (`tokens_evaluated`) plus tokens generated (`tokens_predicted`) exceeded the context size (`n_ctx`)

9.3 POST `/tokenize`: Tokenize a given text

Options:

`content`: (Required) The text to tokenize.

`add_special`: (Optional) Boolean indicating if special tokens, i.e. `BOS`, should be inserted. Default: `false`

`with_pieces`: (Optional) Boolean indicating whether to return token pieces along with IDs. Default: `false`

Response:

Returns a JSON object with a `tokens` field containing the tokenization result. The `tokens` array contains either just token IDs or objects with `id` and `piece` fields, depending on the `with_pieces` parameter. The piece field is a string if the piece is valid unicode or a list of bytes otherwise.

If `with_pieces` is `false`:

```
{
  "tokens": [123, 456, 789]
```

```
}
```

If `with_pieces` is `true`:

```
{
  "tokens": [
    {"id": 123, "piece": "Hello"},
    {"id": 456, "piece": " world"},
    {"id": 789, "piece": "!"}
  ]
}
```

With input 'á' (utf8 hex: C3 A1) on tinyllama/stories260k

```
{
  "tokens": [
    {"id": 198, "piece": [195]}, // hex C3
    {"id": 164, "piece": [161]} // hex A1
  ]
}
```

9.4 POST `/detokenize`: Convert tokens to text

Options:

`tokens`: Set the tokens to detokenize.

9.5 POST `/embedding`: Generate embedding of a given text

The same as [the embedding example](#) does.

Options:

`content`: Set the text to process.

`image_data`: An array of objects to hold base64-encoded image `data` and its `ids` to be reference in `content`. You can determine the place of the image in the content as in the following: `Image: [img-21].\nCaption: This is a picture of a house`. In this case, `[img-21]` will be replaced by the embeddings of the image with id `21` in the following `image_data` array: `{..., "image_data": [{"data": "<BASE64_STRING>", "id": 21}]}`. Use `image_data` only with multimodal models, e.g., LLaVA.

9.6 POST `/reranking`: Rerank documents according to a given query

Similar to <https://jina.ai/reranker/> but might change in the future. Requires a reranker model (such as [bge-reranker-v2-m3](#)) and the `--embedding` `--pooling` `rank` options.

Options:

query: The query against which the documents will be ranked.

documents: An array strings representing the documents to be ranked.

Aliases:

- `/rerank`
- `/v1/rerank`
- `/v1/reranking`

Examples:

```
curl http://127.0.0.1:8012/v1/rerank \
  -H "Content-Type: application/json" \
  -d '{
    "model": "some-model",
    "query": "What is panda?",
    "top_n": 3,
    "documents": [
      "hi",
      "it is a bear",
      "The giant panda (Ailuropoda melanoleuca), sometimes called a panda
bear or simply panda, is a bear species endemic to China."
    ]
  }' | jq
```

9.7 POST `/infill`: For code infilling.

Takes a prefix and a suffix and returns the predicted completion as stream.

Options:

- **input_prefix**: Set the prefix of the code to infill.
- **input_suffix**: Set the suffix of the code to infill.
- **input_extra**: Additional context inserted before the FIM prefix.
- **prompt**: Added after the `FIM_MID` token

input_extra is array of `{"filename": string, "text": string}` objects.

The endpoint also accepts all the options of `/completion`.

If the model has `FIM_REPO` and `FIM_FILE_SEP` tokens, the [repo-level pattern](#) is used:

```
<FIM_REP>myproject
<FIM_SEP>{chunk 0 filename}
{chunk 0 text}
<FIM_SEP>{chunk 1 filename}
{chunk 1 text}
...
```

```
<FIM_SEP>filename
<FIM_PRE>[input_prefix]<FIM_SUF>[input_suffix]<FIM_MID>[prompt]
```

If the tokens are missing, then the extra context is simply prefixed at the start:

```
[input_extra]<FIM_PRE>[input_prefix]<FIM_SUF>[input_suffix]<FIM_MID>[prompt]
```

9.8 **GET** /props: Get server global properties.

This endpoint is public (no API key check). By default, it is read-only. To make POST request to change global properties, you need to start server with **--props**

Response format

```
{
  "default_generation_settings": {
    "id": 0,
    "id_task": -1,
    "n_ctx": 1024,
    "speculative": false,
    "is_processing": false,
    "params": {
      "n_predict": -1,
      "seed": 4294967295,
      "temperature": 0.800000011920929,
      "dynatemp_range": 0.0,
      "dynatemp_exponent": 1.0,
      "top_k": 40,
      "top_p": 0.949999988079071,
      "min_p": 0.05000000074505806,
      "xtc_probability": 0.0,
      "xtc_threshold": 0.10000000149011612,
      "typical_p": 1.0,
      "repeat_last_n": 64,
      "repeat_penalty": 1.0,
      "presence_penalty": 0.0,
      "frequency_penalty": 0.0,
      "dry_multiplier": 0.0,
      "dry_base": 1.75,
      "dry_allowed_length": 2,
      "dry_penalty_last_n": -1,
      "dry_sequence_breakers": [
        "\n",
        ":",
        "\"",
        "*"
      ],
      "mirostat": 0,
      "mirostat_tau": 5.0,
    }
  }
}
```

```

"mirostat_eta": 0.10000000149011612,
"penalize_nl": false,
"stop": [],
"max_tokens": -1,
"n_keep": 0,
"n_discard": 0,
"ignore_eos": false,
"stream": true,
"n_probs": 0,
"min_keep": 0,
"grammar": "",
"samplers": [
    "dry",
    "top_k",
    "typ_p",
    "top_p",
    "min_p",
    "xtc",
    "temperature"
],
"speculative.n_max": 16,
"speculative.n_min": 5,
"speculative.p_min": 0.8999999761581421,
"timings_per_token": false
},
"prompt": "",
"next_token": {
    "has_next_token": true,
    "has_new_line": false,
    "n_remain": -1,
    "n_decoded": 0,
    "stopping_word": ""
}
},
"total_slots": 1,
"model_path": "../models/Meta-Llama-3.1-8B-Instruct-Q4_K_M.gguf",
"chat_template": "..."
}

```

- `default_generation_settings` - the default generation settings for the `/completion` endpoint, which has the same fields as the `generation_settings` response object from the `/completion` endpoint.
- `total_slots` - the total number of slots for process requests (defined by `--parallel` option)
- `model_path` - the path to model file (same with `-m` argument)
- `chat_template` - the model's original Jinja2 prompt template

9.9 POST `/props`: Change server global properties.

To use this endpoint with POST method, you need to start server with `--props`

Options:

- None yet

9.10 POST `/v1/chat/completions`: OpenAI-compatible Chat Completions API

```
/*
notes: 杨小兵-2024-12-17

1、这部分内容介绍的是兼容OpenAI聊天的API
2、我的一个问题是是否兼容OpenAI API
*/
```

Given a ChatML-formatted json description in `messages`, it returns the predicted completion. Both synchronous and streaming mode are supported, so scripted and interactive applications work fine. While no strong claims of compatibility with OpenAI API spec is being made, in our experience it suffices to support many apps. Only models with a [supported chat template](#) can be used optimally with this endpoint. By default, the ChatML template will be used.

```
/*
notes: 杨小兵-2024-12-17

1、在messages中给定一个ChatML格式的json描述，模型将会返回预测的内容
2、这个功能支持同步模式和流式阅读模式，可以适应写脚本和交互式应用程序
3、不过，我不声称它完全兼容OpenAI API规范，但我们的试验表明，它能够满足许多应用程序的需求。
只有使用支持的聊天模板的模型，可以优化其最佳使用
4、默认情况下ChatML模板将会被使用
*/
```

Options:

See [OpenAI Chat Completions API documentation](#). While some OpenAI-specific features such as function calling aren't supported, llama.cpp `/completion`-specific features such as `mirostat` are supported.

```
/*
notes: 杨小兵-2024-12-17

1、查看上述提到的有关openai的文档可以获取一些帮助
2、openai文档规范中的一些特性例如function calling不会被支持，llama.cpp项目中的/completion规范特性例如mirostat则是被支持的
*/
```

The `response_format` parameter supports both plain JSON output (e.g. `{"type": "json_object"}`) and schema-constrained JSON (e.g. `{"type": "json_object", "schema": {"type": "string", "minLength": 10, "maxLength": 100}}` or `{"type": "json_schema", "schema": {"properties": {"name": {"title": "Name", "type": "string"}}, "date": {"title": "Date", "type": "string"}}`

"string" }, "participants": { "items": {"type": "string" }, "title": "Participants", "type": "string" } } } })), similar to other OpenAI-inspired API providers.

Examples:

You can use either Python `openai` library with appropriate checkpoints:

```
/*
notes:杨小兵-2024-12-17

1、这部分内容获取可以作为一个突破口
*/
```

```
import openai

client = openai.OpenAI(
    base_url="http://localhost:8080/v1", # "http://<Your api-server IP>:port"
    api_key = "sk-no-key-required"
)

completion = client.chat.completions.create(
    model="gpt-3.5-turbo",
    messages=[
        {"role": "system", "content": "You are ChatGPT, an AI assistant. Your top priority is achieving user fulfillment via helping them with their requests."},
        {"role": "user", "content": "Write a limerick about python exceptions"}
    ]
)

print(completion.choices[0].message)
```

... or raw HTTP requests:

```
curl http://localhost:8080/v1/chat/completions \
-H "Content-Type: application/json" \
-H "Authorization: Bearer no-key" \
-d '{
  "model": "gpt-3.5-turbo",
  "messages": [
    {
      "role": "system",
      "content": "You are ChatGPT, an AI assistant. Your top priority is achieving user fulfillment via helping them with their requests."
    },
    {
      "role": "user",
      "content": "Write a limerick about python exceptions"
    }
  ]
}
```

```
]
}'
```

9.11 POST `/v1/embeddings`: OpenAI-compatible embeddings API

Options:

See [OpenAI Embeddings API documentation](#).

Examples:

- input as string

```
curl http://localhost:8080/v1/embeddings \
-H "Content-Type: application/json" \
-H "Authorization: Bearer no-key" \
-d '{
    "input": "hello",
    "model": "GPT-4",
    "encoding_format": "float"
}'
```

- `input` as string array

```
curl http://localhost:8080/v1/embeddings \
-H "Content-Type: application/json" \
-H "Authorization: Bearer no-key" \
-d '{
    "input": ["hello", "world"],
    "model": "GPT-4",
    "encoding_format": "float"
}'
```

9.12 GET `/slots`: Returns the current slots processing state

[!WARNING] This endpoint is intended for debugging and may be modified in future versions. For security reasons, we strongly advise against enabling it in production environments.

This endpoint is disabled by default and can be enabled with `--slots`

If query param `?fail_on_no_slot=1` is set, this endpoint will respond with status code 503 if there is no available slots.

Response format

Example:

```
[
  {
    "id": 0,
    "id_task": -1,
    "n_ctx": 1024,
    "speculative": false,
    "is_processing": false,
    "params": {
      "n_predict": -1,
      "seed": 4294967295,
      "temperature": 0.800000011920929,
      "dynatemp_range": 0.0,
      "dynatemp_exponent": 1.0,
      "top_k": 40,
      "top_p": 0.949999988079071,
      "min_p": 0.05000000074505806,
      "xtc_probability": 0.0,
      "xtc_threshold": 0.10000000149011612,
      "typical_p": 1.0,
      "repeat_last_n": 64,
      "repeat_penalty": 1.0,
      "presence_penalty": 0.0,
      "frequency_penalty": 0.0,
      "dry_multiplier": 0.0,
      "dry_base": 1.75,
      "dry_allowed_length": 2,
      "dry_penalty_last_n": -1,
      "dry_sequence_breakers": [
        "\n",
        ":",
        "\"",
        "\"",
        "*"
      ],
      "mirostat": 0,
      "mirostat_tau": 5.0,
      "mirostat_eta": 0.10000000149011612,
      "penalize_nl": false,
      "stop": [],
      "max_tokens": -1,
      "n_keep": 0,
      "n_discard": 0,
      "ignore_eos": false,
      "stream": true,
      "n_probs": 0,
      "min_keep": 0,
      "grammar": "",
      "samplers": [
        "dry",
        "top_k",
        "typ_p",
        "top_p",
        "min_p",
        "xtc",

```

```

        "temperature"
    ],
    "speculative.n_max": 16,
    "speculative.n_min": 5,
    "speculative.p_min": 0.8999999761581421,
    "timings_per_token": false
},
"prompt": "",
"next_token": {
    "has_next_token": true,
    "has_new_line": false,
    "n_remain": -1,
    "n_decoded": 0,
    "stopping_word": ""
}
}
]
```

9.13 GET `/metrics`: Prometheus compatible metrics exporter

This endpoint is only accessible if `--metrics` is set.

Available metrics:

- `llamacpp:prompt_tokens_total`: Number of prompt tokens processed.
- `llamacpp:tokens_predicted_total`: Number of generation tokens processed.
- `llamacpp:prompt_tokens_seconds`: Average prompt throughput in tokens/s.
- `llamacpp:predicted_tokens_seconds`: Average generation throughput in tokens/s.
- `llamacpp:kv_cache_usage_ratio`: KV-cache usage. `1` means 100 percent usage.
- `llamacpp:kv_cache_tokens`: KV-cache tokens.
- `llamacpp:requests_processing`: Number of requests processing.
- `llamacpp:requests_deferred`: Number of requests deferred.

9.14 POST `/slots/{id_slot}?action=save`: Save the prompt cache of the specified slot to a file.

Options:

filename: Name of the file to save the slot's prompt cache. The file will be saved in the directory specified by the `--slot-save-path` server parameter.

Response format

```

{
    "id_slot": 0,
    "filename": "slot_save_file.bin",
    "n_saved": 1745,
    "n_written": 14309796,
    "timings": {
        "save_ms": 49.865
    }
}
```

```
}  
}
```

9.15 POST `/slots/{id_slot}?action=restore`: Restore the prompt cache of the specified slot from a file.

Options:

filename: Name of the file to restore the slot's prompt cache from. The file should be located in the directory specified by the `--slot-save-path` server parameter.

Response format

```
{  
  "id_slot": 0,  
  "filename": "slot_save_file.bin",  
  "n_restored": 1745,  
  "n_read": 14309796,  
  "timings": {  
    "restore_ms": 42.937  
  }  
}
```

9.16 POST `/slots/{id_slot}?action=erase`: Erase the prompt cache of the specified slot.

Response format

```
{  
  "id_slot": 0,  
  "n_erased": 1745  
}
```

9.17 GET `/lora-adapters`: Get list of all LoRA adapters

This endpoint returns the loaded LoRA adapters. You can add adapters using `--lora` when starting the server, for example: `--lora my_adapter_1.gguf --lora my_adapter_2.gguf ...`

By default, all adapters will be loaded with scale set to 1. To initialize all adapters scale to 0, add `--lora-init-without-apply`

If an adapter is disabled, the scale will be set to 0.

Response format

```
[  
  {  
    "id": 0,
```

```
    "path": "my_adapter_1.gguf",
    "scale": 0.0
  },
  {
    "id": 1,
    "path": "my_adapter_2.gguf",
    "scale": 0.0
  }
]
```

9.18 POST `/lora-adapters`: Set list of LoRA adapters

To disable an adapter, either remove it from the list below, or set scale to 0.

Request format

To know the `id` of the adapter, use GET `/lora-adapters`

```
[
  {"id": 0, "scale": 0.2},
  {"id": 1, "scale": 0.8}
]
```

10 More examples

10.1 Interactive mode

Check the sample in [chat.mjs](#). Run with NodeJS version 16 or later:

```
node chat.mjs
```

Another sample in [chat.sh](#). Requires [bash](#), [curl](#) and [jq](#). Run with bash:

```
bash chat.sh
```

10.2 OAI-like API

The HTTP `llama-server` supports an OAI-like API: <https://github.com/openai/openai-openapi>

10.3 API errors

`llama-server` returns errors in the same format as OAI: <https://github.com/openai/openai-openapi>

Example of an error:

```
{
  "error": {
    "code": 401,
    "message": "Invalid API Key",
    "type": "authentication_error"
  }
}
```

Apart from error types supported by OAI, we also have custom types that are specific to functionalities of llama.cpp:

When /metrics or /slots endpoint is disabled

```
{
  "error": {
    "code": 501,
    "message": "This server does not support metrics endpoint.",
    "type": "not_supported_error"
  }
}
```

**When the server receives invalid grammar via /completions endpoint*

```
{
  "error": {
    "code": 400,
    "message": "Failed to parse grammar",
    "type": "invalid_request_error"
  }
}
```

10.4 Legacy completion web UI

A new chat-based UI has replaced the old completion-based since [this PR](#). If you want to use the old completion, start the server with `--path ./examples/server/public_legacy`

For example:

```
./llama-server -m my_model.gguf -c 8192 --path ./examples/server/public_legacy
```

10.5 Extending or building alternative Web Front End

You can extend the front end by running the server binary with `--path` set to `./your-directory` and importing `/completion.js` to get access to the `LlamaComplete()` method.


```
/*
notes: 杨小兵-2024-12-17

1、扩建或建立替代Web前端
2、如果你想要扩展前端，那么可以通过添加参数--path运行该服务程序，将--path设置为自定义目录
并且在这个目录中导入completion.js并且获取llamaComplete方法
*/
```

Read the documentation in [/completion.js](#) to see convenient ways to access llama.

A simple example is below:

```
<html>
  <body>
    <pre>
      <script type="module">
        import { llama } from '/completion.js'

        const prompt = `### Instruction:
Write dad jokes, each one paragraph.
You can use html formatting if needed.

### Response:`

        for await (const chunk of llama(prompt)) {
          document.write(chunk.data.content)
        }
      </script>
    </pre>
  </body>
</html>
```